BBM 418 & AIN 433 - Computer Vision Laboratory

HACETTEPE UNIVERSITY

Programming Assignment-3
Fall 2025-2026
Assoc. Prof. Dr. Ali Seydi KEÇELİ
TAs: *M. Aslı Taşgetiren* & S. Meryem Taşyürek

Due Date: 23:59 on Wednesday, 9 December 2025

Image Colorization Using Deep Convolutional Neural Networks

In this assignment, we will focus on image colorization using Deep Convolutional Neural Networks (DCNNs). For this task, encoder-decoder architectures are well-known and widely used methods. These architectures are particularly effective for learning complex mappings between grayscale images and their corresponding colorized versions.

Image colorization involves adding realistic colors to black-and-white (grayscale) photographs or videos. This technique is often employed to restore and enhance old photographs, historical documents, and vintage films, breathing new life into them and making them more vivid and engaging for modern audiences.

Figure 1: ~~Image colorization examples with DCCNs.~~

L*a*b* Color Model

The L*a*b* color model, or CIELAB, represents color as three components: L* for lightness (0 for black to 100 for white), a* for the green-to-red spectrum, and b* for the blue-to-yellow spectrum. It is perceptually uniform, meaning value differences align with human perception, and separates brightness (L*) from color information (a*, b*). This separation makes the colorization problem natural: input the L channel (grayscale) and predict the a,b channels. An example for L*a*b* color model channels is given on Figure 2.

Figure 2: An example for L*a*b* color model.

Dataset

You are expected to build your own dataset with at least 5000 color images. For each color image:

- Save a color version (ground truth).
- Save a grayscale (L channel) version. Convert RGB to CIELAB and use L as input and a/b as targets.

Important: Save your dataset to Drive and continue working from there, i.e., load your images from Drive instead of downloading them and using your local storage. You must also share the Drive folder with me as well so that I can run your notebook same way without downloading any image files.

Base Encoder-Decoder Architecture

The model (Figure 3) follows an encoder-decoder architecture designed for image colorization using the L*a*b* color space. It begins by taking a grayscale image representing the L (luminance) channel and passing it through two feature extraction networks: a low-level feature extraction network for local spatial features and a global feature extraction network for broader context. These features are then fused into a single vector, which is sent to the decoder. The decoder progressively upsamples the compressed representation, restoring the image's spatial resolution and predicting the missing a* and b* chrominance channels that encode color information. The output is a fully colorized image, where the original luminance (L) is combined with the predicted chrominance channels (a* and b*), producing a colorized image in the L*a*b* space.

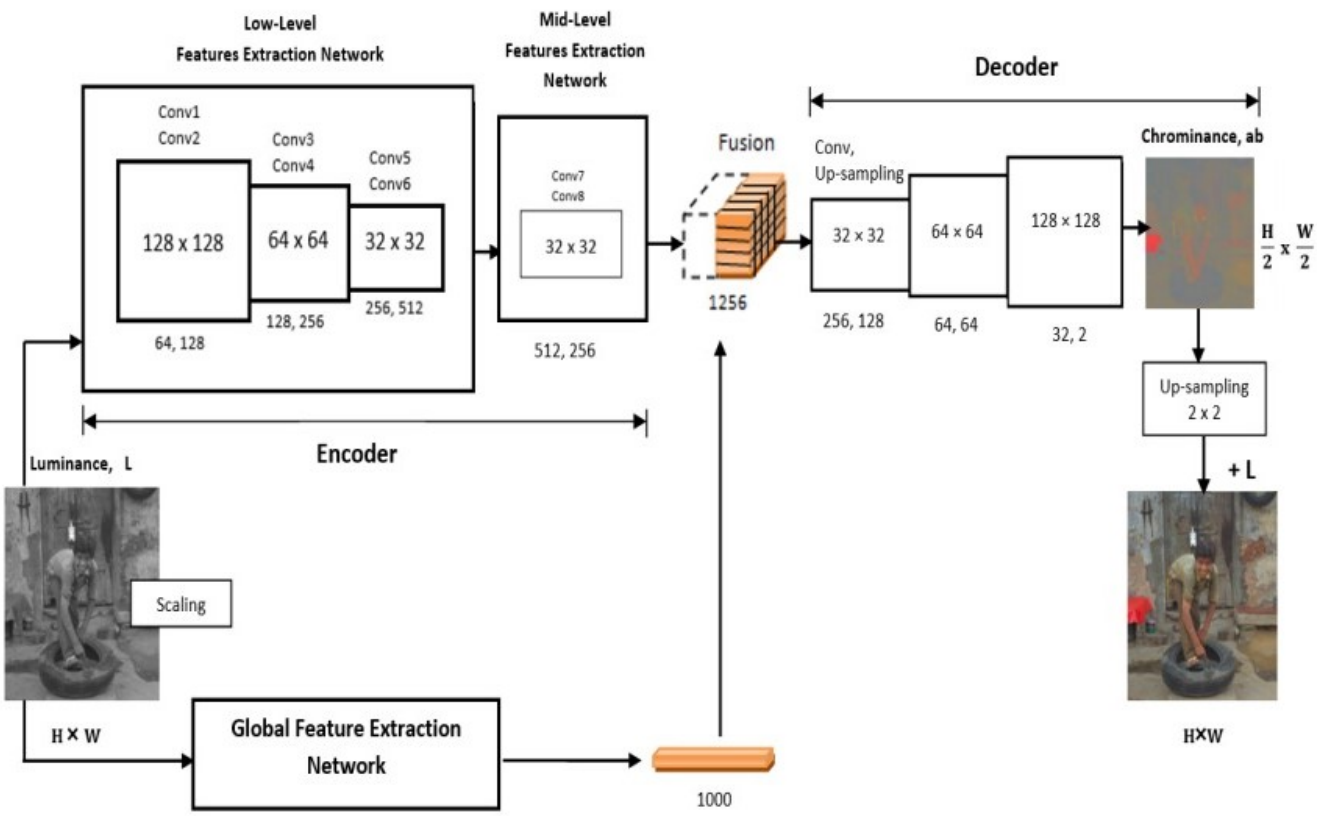

Figure 3: Base encoder–decoder architecture. Input: L channel (256×256×1).

Note: This figure describes the *base* architecture that you should use as a starting point. You should extend the design by experimenting with different global feature extractors (e.g., VGG, ResNet, EfficientNet), modifying the encoder/decoder blocks (skip connections, attention, more filters), adding perceptual loss, or integrating a PatchGAN discriminator as an optional extension. Make small experiments first, then scale up.

Implementation Details

1. **Create your dataset** and save it to the Google Drive
2. **Split the Dataset**
   - Use **80% for training**.
   - Use **20% for testing/validation**.
   - Ensure shuffling is enabled.
3. **Implement the Baseline Encoder–Decoder Model**
   - Use PyTorch to implement the architecture shown in Figure 3 (local feature extractor, global feature extractor, fusion block, decoder).
   - Train this baseline once using:
     - a pixel-wise loss (L1 or MSE),
     - an optimizer of your choice (Adam recommended).

   This model becomes your **reference** for comparison.
4. **Integrate A Pretrained Global Feature Extractor**

   Integrate at least one pretrained global feature extractor. You may try more for extra comparison, but only one is required. For example, you may choose from:

   - VGG (e.g., VGG16)
   - ResNet (e.g., ResNet50)
   - EfficientNet (e.g., EfficientNet-B0)
   - MobileNet (e.g., MobileNetV2)
   - (or any other pre-trained backbone)

   For each selected backbone:

   (a) replace the original global feature extractor with that backbone,
   (b) train the model,
   (c) evaluate (MSE, PSNR, SSIM),
   (d) show sample colorized outputs,
   (e) compare visually and numerically.

   Each backbone (if more than one, one is enough) should be tested in its own separate model configuration. You may fine-tune or freeze the backbone.
5. **Improve Model Architecture**

   Modify the encoder–decoder architecture as necessary to improve colorization results. Example improvements might include adding skip connections, changing the number of layers, or modifying the fusion strategy. *Explain and justify all architectural choices.*
6. **Implement Perceptual Loss**

   Use a pretrained VGG network (or similar) to compute a feature-based loss between the predicted color image and the ground-truth image, encouraging more realistic colors and textures.

   You can compare this with L1 loss, which measures the absolute pixel-wise difference between the predicted and ground-truth color channels. L1 loss is commonly used in image-to-image tasks because it encourages smoother and more stable predictions.

   You can compare:

   - using only L1 loss,
   - using only perceptual loss,
   - using both together.

   And discuss how images look different.

7. **Bonus (10 points): PatchGAN Discriminator for Decoder Output**

   As a bonus add a PatchGAN discriminator after the decoder.

   PatchGAN is a small convolutional network that does not classify the whole image as "real" or "fake." Instead, it looks at many *small patches* of the image (e.g., 70×70 regions) and judges whether each patch looks realistic. This helps the model produce sharper and more locally consistent colors.

   The discriminator should compare:

   - the decoder's colorized output, and
   - the ground-truth color image.

   The discriminator's PatchGAN loss is then used to further train the decoder:

   - Use the PatchGAN adversarial loss,
   - optionally combine it with L1 or perceptual loss.

   This setup does **not** turn the entire model into a full cGAN; instead, it only adds a PatchGAN module that evaluates the decoder's output and guides it toward producing more realistic colorizations.

   Compare the results of:

   - decoder trained without PatchGAN,
   - decoder trained with PatchGAN.

Evaluate Your Models

Evaluate your model using, Mean Squared Error (MSE), Peak Signal-to-Noise Ratio (PSNR) and Structural Similarity Index (SSIM). Compare different architectures, feature extractors, and loss configurations.

Visualization

Plot the training and testing curves (e.g., loss and MSE per epoch). Provide several example colorized outputs similar to Figure 1. Include comparisons between different models and experimental settings.

Implementation Tip: You can use Google Colab if your system does not have a suitable GPU.

Demo Video (Mandatory, Maximum 5 Minutes)

Submit a short presentation video (`demo.mp4`) summarizing your work (as a Drive or private Youtube video link).

Your video should:

- briefly explain the goal of the assignment and your pipeline (dataset, model, losses, experiments),
- show selected figures and visuals from your notebook/report (sample outputs, comparison tables, metrics),
- demonstrate that your code runs (short snippets are enough),
- highlight important implementation decisions,
- present your best results clearly.

You can talk about similar things in your report and the video; however, the video should be used to walk through your Jupyter Notebook and you **must not simply read out the report sections**. Focus on showing and explaining, not reading. See next section for details about report parts of your notebook.

The video must be clear, self-contained, and under 5 minutes. You may use Zoom, OBS, PowerPoint screen recording, or similar tools to record your video.

What should you write in the report?

The notebook should include:

1. Title, name, student number, dataset path variables, link to demo video,
2. Brief introduction to the problem,
3. Dataset preparation (include a few sample images and L/a/b visualizations),
4. Model definitions (low-level encoder, global extractor variants, fusion, decoder),
5. Final test results and selected qualitative examples,
6. A summary table comparing all models,
7. Visual samples from each models,
8. Discussion of:
   - which version of the model performed best,
   - which loss function helped,
   - effect of pretrained global extractors,
   - whether perceptual loss improved visual quality,
   - training time and computational cost.
9. Failure cases and short discussion,
10. Short summary and suggestions for improvement.

Submission

Prepare a ZIP file named `b<student_number>.zip` containing:

- `assignment_3.ipynb` — the Jupyter notebook (**well-commented code + report**)

Submit the ZIP via the Hacettepe Submit system.

Grading

Total: **100 points**

- **Code correctness and completeness (80 pts)**:
  - 25 pts — Baseline encoder–decoder implemented, training loop works, and data pipeline (RGB→Lab, L input, ab target) is correct.
  - 15 pts — Pretrained global feature extractor (e.g., VGG, ResNet, EfficientNet) integrated and compared.
  - 10 pts — Perceptual loss implemented correctly.
  - 10 pts — Code quality: modular structure, clear functions, comments, reproducibility (dataset path, random seed, etc.).
  - **20 pts — You *will be* graded on how well your models work and how good your results are.**
- **Report and demo video (20 pts)**:
  - 10 pts — Presentation of results, including correct computation of MSE, PSNR, SSIM, tables summarizing model performance, and plots of losses or training curves.
  - 5 pts — Qualitative results: at least 5 visual comparisons (like Figure 1) (input L, ground truth, prediction) + failure cases.
  - 5 pts — Clear explanation of methods, comparison tables, and final conclusions.

Note: Preparing a good report is important as well as the correctness of your solutions! You should explain your choices and their effects on the results. Create a table/tables to report your results.

Bonus (up to 10 points): For adding a PatchGAN discriminator after the decoder output, training it properly, and showing meaningful visual improvement.

Academic Integrity

All work on assignments must be done individually unless stated otherwise. You are encouraged to discuss with your classmates about the given assignments, but these discussions should be carried out in an abstract way. That is, discussions related to a particular solution to a specific problem (either in actual code or in the pseudocode) will not be tolerated. In short, turning in someone else's work, in whole or in part, as your own will be considered as a violation of academic integrity.